

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №4 по курсу
«Операционные системы»

Группа: М8О-213Б-23

Студент: Зайтова Е.А

Преподаватель: Бахарев В.Д. (ФИИТ)

Оценка: _____

Дата: 27.12.24

Москва, 2024

Постановка задачи

Вариант 5.

Исследовать два аллокатора памяти: необходимо реализовать два алгоритма

аллокации памяти и сравнить их по следующим характеристикам:

- Фактор использования
- Скорость выделения блоков
- Скорость освобождения блоков
- Простота использования аллокатора

5. Алгоритм Мак-Кьюзика-Кэрелса и алгоритм двойников;

Общий метод и алгоритм решения

Использованные системные вызовы:

- write — для вывода сообщений на стандартные потоки (STDOUT и STDERR).
- mmap — для выделения блока виртуальной памяти.
- munmap — для освобождения ранее выделенной виртуальной памяти

Реализуем аллокатор для алгоритма двойников – Buddy Allocator, для Мак-Кьюзика-Кэрелса – Fixed-Size Allocator.

Оба аллокатора реализуют общий интерфейс через функции `allocator_create`, `allocator_alloc`, `allocator_free` и `allocator_destroy`. Buddy Allocator организует память в виде бинарного дерева (BlockNode), где каждый узел представляет блок памяти. При выделении памяти блок делится на два, пока его размер не станет равен запрашиваемому. Функция `allocator_alloc` рекурсивно ищет подходящий блок, а при освобождении (`allocator_free`) объединяет соседние свободные блоки. Этот метод оптимален для работы с блоками разного размера.

Fixed-Size Allocator использует массив списков свободных блоков (FreeBlock), где каждый список отвечает за блоки фиксированного размера, кратного степени двойки (16, 32, 64 и т.д.). Функция `allocator_alloc` ищет первый свободный блок в соответствующем списке, а если список пуст, выделяет новый блок. Освобождённый блок через `allocator_free` возвращается в список. Такой подход эффективен для повторяющихся запросов одного размера. Основная программа через `load_allocator` загружает библиотеку с одним из аллокаторов (через `dlopen`) и вызывает их функции. Реализуется проверка работы: выделяются и освобождаются блоки памяти, а аллокатор уничтожается через `allocator_destroy`. Работа обеих реализаций тестируется с помощью общего интерфейса.

Код программы

main.c

```
#include <stddef.h>

#include <stdlib.h>

#include <stdint.h>

#include <string.h>

#include <stdio.h>

#include <unistd.h>

#include <dlfcn.h>

#include <sys/mman.h>
```

```
typedef struct Allocator {

    void* (*allocator_create)(void* addr, size_t size);

    void* (*allocator_alloc)(void* allocator, size_t size);

    void (*allocator_free)(void* allocator, void* ptr);

    void (*allocator_destroy)(void* allocator);

} Allocator;
```

```
void* standard_allocator_create(void* memory, size_t size) {

    (void)size;

    (void)memory;

    return memory;

}
```

```
void* standard_allocator_alloc(void* allocator, size_t size) {

    uint32_t* memory = mmap(NULL, size + sizeof(uint32_t), PROT_READ | PROT_WRITE,

        MAP_SHARED | MAP_ANONYMOUS, -1, 0);

    if (memory == MAP_FAILED) {

        return NULL;

    }

    *memory = (uint32_t)(size + sizeof(uint32_t));

    return memory + 1;

}
```

```
void standard_allocator_free(void* allocator, void* memory) {

    if (memory == NULL) {
```

```

    return;
}

uint32_t* mem = (uint32_t*)memory - 1;

munmap(mem, *mem);
}

```

```

void standard_allocator_destroy(void* allocator) {
    (void)allocator;
}

```

```

Allocator* load_allocator(const char* library_path) {
    if (library_path == NULL || library_path[0] == '\0') {
        char message[] = "WARNING: failed to load shared library\n";
        write(STDERR_FILENO, message, sizeof(message) - 1);

        Allocator* allocator = malloc(sizeof(Allocator));

        allocator->allocator_create = standard_allocator_create;
        allocator->allocator_alloc = standard_allocator_alloc;
        allocator->allocator_free = standard_allocator_free;
        allocator->allocator_destroy = standard_allocator_destroy;

        return allocator;
    }
}

```

```

void* library = dlopen(library_path, RTLD_LOCAL | RTLD_NOW);
if (!library) {
    char message[] = "WARNING: failed to load shared library\n";
    const char* error_message = dlerror();

    fprintf(stderr, "ERROR: dlopen failed: %s\n", error_message);
    write(STDERR_FILENO, message, sizeof(message) - 1);

    Allocator* allocator = malloc(sizeof(Allocator));

    allocator->allocator_create = standard_allocator_create;
    allocator->allocator_alloc = standard_allocator_alloc;
    allocator->allocator_free = standard_allocator_free;
    allocator->allocator_destroy = standard_allocator_destroy;

    return allocator;
}

```

```
}
```

```
Allocator* allocator = malloc(sizeof(Allocator));
```

```
allocator->allocator_create = dlsym(library, "allocator_create");
```

```
allocator->allocator_alloc = dlsym(library, "allocator_alloc");
```

```
allocator->allocator_free = dlsym(library, "allocator_free");
```

```
allocator->allocator_destroy = dlsym(library, "allocator_destroy");
```

```
if (!allocator->allocator_create || !allocator->allocator_alloc ||
```

```
    !allocator->allocator_free || !allocator->allocator_destroy) {
```

```
    const char msg[] = "Error: failed to load all allocator functions\n";
```

```
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
```

```
    free(allocator);
```

```
    dlclose(library);
```

```
    return NULL;
```

```
}
```

```
return allocator;
```

```
}
```

```
int main(int argc, char** argv) {
```

```
    const char* library_path = (argc > 1) ? argv[1] : NULL;
```

```
    Allocator* allocator_api = load_allocator(library_path);
```

```
    if (!allocator_api) {
```

```
        char message[] = "Failed to load allocator API\n";
```

```
        write(STDERR_FILENO, message, sizeof(message) - 1);
```

```
        return EXIT_FAILURE;
```

```
}
```

```
size_t size = 4096;
```

```
void* addr = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
```

```
if (addr == MAP_FAILED) {
```

```
    char message[] = "mmap failed\n";
```

```
write(STDERR_FILENO, message, sizeof(message) - 1);

free(allocator_api);

return EXIT_FAILURE;

}
```

```
void* allocator = allocator_api->allocator_create(addr, size);

if (!allocator) {

    char message[] = "Failed to initialize allocator\n";

    write(STDERR_FILENO, message, sizeof(message) - 1);

    munmap(addr, size);

    free(allocator_api);

    return EXIT_FAILURE;

}
```

```
char start_message[] = "Allocator initialized\n";

write(STDOUT_FILENO, start_message, sizeof(start_message) - 1);
```

```
void* block1 = allocator_api->allocator_alloc(allocator, 64);

void* block2 = allocator_api->allocator_alloc(allocator, 40);
```

```
if (block1 == NULL || block2 == NULL) {

    char alloc_fail_message[] = "Memory allocation failed\n";

    write(STDERR_FILENO, alloc_fail_message, sizeof(alloc_fail_message) - 1);

} else {

    char alloc_success_message[] = "Memory allocated successfully\n";

    write(STDOUT_FILENO, alloc_success_message, sizeof(alloc_success_message) - 1);

}
```

```
char buffer[64];

snprintf(buffer, sizeof(buffer), "Block 1 address: %p\n", block1);

write(STDOUT_FILENO, buffer, strlen(buffer));

snprintf(buffer, sizeof(buffer), "Block 2 address: %p\n", block2);

write(STDOUT_FILENO, buffer, strlen(buffer));
```

```

allocator_api->allocator_free(allocator, block1);
allocator_api->allocator_free(allocator, block2);

char free_message[] = "Memory freed\n";
write(STDOUT_FILENO, free_message, sizeof(free_message) - 1);

allocator_api->allocator_destroy(allocator);
free(allocator_api);
munmap(addr, size);

char exit_message[] = "Program exited successfully\n";
write(STDOUT_FILENO, exit_message, sizeof(exit_message) - 1);

return EXIT_SUCCESS;
}

```

Buddy_allocator.c

```

#include <stdlib.h>
#include <unistd.h>
#include <math.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

typedef struct BlockNode {
    size_t block_size;
    int is_free;
    struct BlockNode* left;
    struct BlockNode* right;
} BlockNode;

typedef struct Allocator {
    BlockNode* root;
    void* memory_start;

```

```
    size_t total_memory;

    size_t current_position;
} Allocator;
```

```
int is_power_of_two(size_t n) {
    if (n == 0) {
        return 0;
    }
    while (n > 1) {
        if (n % 2 != 0) {
            return 0;
        }
        n /= 2;
    }
    return 1;
}
```

```
BlockNode* create_block_node(Allocator* allocator, size_t block_size) {
    if (allocator->current_position + sizeof(BlockNode) > allocator->total_memory) {
        return NULL;
    }
}
```

```
BlockNode* node = (BlockNode*)((char*)allocator->memory_start + allocator->current_position);
allocator->current_position += sizeof(BlockNode);
node->block_size = block_size;
node->is_free = 1;
node->left = node->right = NULL;
return node;
}
```

```
Allocator* allocator_create(void* memory, const size_t size) {
    if (!is_power_of_two(size)) {
        const char* error_msg = "Allocator requires memory size to be a power of two\n";
        write(STDERR_FILENO, error_msg, strlen(error_msg));
    }
}
```

```
    return NULL;
}
```

```
Allocator* allocator = (Allocator*)memory;
allocator->memory_start = (char*)memory + sizeof(Allocator);
allocator->total_memory = size - sizeof(Allocator);
allocator->current_position = 0;
```

```
allocator->root = create_block_node(allocator, size);
if (!allocator->root) {
    return NULL;
}
```

```
return allocator;
}
```

```
void split_block(Allocator* allocator, BlockNode* node) {
    size_t half_size = node->block_size / 2;

    node->left = create_block_node(allocator, half_size);
    node->right = create_block_node(allocator, half_size);
}
```

```
BlockNode* recursive_allocation(Allocator* allocator, BlockNode* node, size_t size) {
    if (!node || node->block_size < size || !node->is_free) {
        return NULL;
    }
```

```
    if (node->block_size == size) {
        node->is_free = 0;
        return node;
    }
```

```
    if (!node->left) {
```

```

    split_block(allocator, node);
}

BlockNode* allocated_node = recursive_allocation(allocator, node->left, size);
if (!allocated_node) {
    allocated_node = recursive_allocation(allocator, node->right, size);
}

node->is_free = (node->left && node->left->is_free) || (node->right && node->right->is_free);
return allocated_node;
}

void* allocator_alloc(Allocator* allocator, const size_t size) {
    if (!allocator || size == 0) {
        return NULL;
    }

    size_t aligned_size = size;
    while (!is_power_of_two(aligned_size)) {
        aligned_size++;
    }

    BlockNode* block = recursive_allocation(allocator, allocator->root, aligned_size);
    return block ? (void*)block : NULL;
}

void allocator_free(Allocator* allocator, void* memory) {
    if (!allocator || !memory) {
        return;
    }

    BlockNode* block = (BlockNode*)memory;
    block->is_free = 1;

```

```

    if (block->left && block->left->is_free && block->right->is_free) {

        allocator_free(allocator, block->left);

        allocator_free(allocator, block->right);

        block->left = block->right = NULL;

    }
}

void allocator_destroy(Allocator* allocator) {

    if (!allocator) {

        return;

    }

    allocator_free(allocator, allocator->root);

    if (munmap(allocator, allocator->total_memory + sizeof(Allocator)) != 0) {

        const char* error_msg = "Failed to unmap memory\n";

        write(STDERR_FILENO, error_msg, strlen(error_msg));

        exit(EXIT_FAILURE);

    }

}

```

fixed size allocator.c

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>


#define MIN_BLOCK_SIZE 16

#define MAX_BLOCK_SIZE 1024


typedef struct FreeBlock {

    struct FreeBlock* next;

} FreeBlock;


typedef struct Allocator {

    FreeBlock* free_lists[10];

    void* memory_start;

```

```

    size_t total_memory;
} Allocator;

int get_free_list_index(size_t size) {
    int index = 0;
    size_t current_size = MIN_BLOCK_SIZE;

    while (current_size < size && current_size <= MAX_BLOCK_SIZE) {
        current_size *= 2;
        index++;
    }

    return index;
}

Allocator* allocator_create(void* const memory, const size_t size) {
    if (size < MIN_BLOCK_SIZE || size > MAX_BLOCK_SIZE) {
        fprintf(stderr, "Error: Memory size must be between %d and %d bytes.\n",
            MIN_BLOCK_SIZE, MAX_BLOCK_SIZE);
        return NULL;
    }

    Allocator* allocator = (Allocator*)memory;
    allocator->memory_start = (char*)memory + sizeof(Allocator);
    allocator->total_memory = size - sizeof(Allocator);

    memset(allocator->free_lists, 0, sizeof(allocator->free_lists));
    return allocator;
}

void allocator_destroy(Allocator* const allocator) {
    if (!allocator) {
        return;
    }
}

```

```

allocator->memory_start = NULL;

allocator->total_memory = 0;


memset(allocator->free_lists, 0, sizeof(allocator->free_lists));
}


void* allocator_alloc(Allocator* const allocator, const size_t size) {
    if (!allocator || size == 0 || size > MAX_BLOCK_SIZE) {
        return NULL;
    }

    int index = get_free_list_index(size);

    if (!allocator->free_lists[index]) {
        size_t block_size = MIN_BLOCK_SIZE << index;
        return malloc(block_size);
    }

    FreeBlock* block = allocator->free_lists[index];
    allocator->free_lists[index] = block->next;

    return (void*)block;
}


void allocator_free(Allocator* const allocator, void* const memory) {
    if (!allocator || !memory) {
        return;
    }

    size_t size = ((FreeBlock*)memory)->next ? MAX_BLOCK_SIZE : MIN_BLOCK_SIZE;
    int index = get_free_list_index(size);

    FreeBlock* block = (FreeBlock*)memory;
    block->next = allocator->free_lists[index];

```

```
allocator->free_lists[index] = block;  
}
```

} **Протокол работы программы**

Тестирование

gcc main.c

\$./a.out

WARNING: failed to load shared library

Allocator initialized

Memory allocated successfully

Block 1 address: 0x7fdd7d80e004

Block 2 address: 0x7fdd7d80d004

Memory freed

Program exited successfully

\$./a.out ./buddy_allocator.so

Allocator initialized

Memory allocated successfully

Block 1 address: 0x7f28b07fc180

Block 2 address: 0x7f28b07fc1a0

Memory freed

Program exited successfully

./a.out ./fixed_size_allocator.so

Allocator initialized

Memory allocated successfully

Block 1 address: 0x55cfc7240240

Block 2 address: 0x55cfc7240290

Memory freed

Program exited successfully

Strace

execve("./a.out", ["./a.out"], 0x7ffeb41a4ce0 /* 27 vars */) = 0

brk(NULL) = 0x55bd1a085000

arch_prctl(0x3001 /* ARCH_??? */, 0x7ffe10bb29a0) = -1 EINVAL (Invalid argument)

mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f1e75830000

access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=18367, ...}, AT_EMPTY_PATH) = 0

mmap(NULL, 18367, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f1e7582b000

close(3) = 0

openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0"..., 832) = 832

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784

pread64(3, "\4\0\0\0\0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0"..., 48, 848) = 48

pread64(3, "\4\0\0\0\24\0\0\0\3\0\0\0GNU\0I\17\357\204\3\$\f\221\2039x\324\224\323\236S"..., 68, 896) = 68

newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...}, AT_EMPTY_PATH) = 0

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784

mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f1e75602000

mprotect(0x7f1e7562a000, 2023424, PROT_NONE) = 0

mmap(0x7f1e7562a000, 1658880, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7f1e7562a000

mmap(0x7f1e757bf000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1bd000) = 0x7f1e757bf000

mmap(0x7f1e75818000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x215000) = 0x7f1e75818000

mmap(0x7f1e7581e000, 52816, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f1e7581e000

close(3) = 0

mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f1e755ff000

arch_prctl(ARCH_SET_FS, 0x7f1e755ff740) = 0

set_tid_address(0x7f1e755ffa10) = 10763

set_robust_list(0x7f1e755ffa20, 24) = 0

```

rseq(0x7f1e756000e0, 0x20, 0, 0x53053053) = 0
mprotect(0x7f1e75818000, 16384, PROT_READ) = 0
mprotect(0x55bd00dcc000, 4096, PROT_READ) = 0
mprotect(0x7f1e7586a000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY})
= 0

munmap(0x7f1e7582b000, 18367) = 0

write(2, "WARNING: failed to load shared l"..., 39WARNING: failed to load shared library
) = 39

getrandom("\xf5\xe7\xe8\x8a\x07\x60\x69\xee", 8, GRND_NONBLOCK) = 8

brk(NULL) = 0x55bd1a085000

brk(0x55bd1a0a6000) = 0x55bd1a0a6000

mmap(NULL, 512, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -
1, 0) = 0x7f1e75869000

write(1, "Allocator initialized\n", 22Allocator initialized
) = 22

mmap(NULL, 68, PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS, -1,
0) = 0x7f1e7582f000

mmap(NULL, 44, PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS, -1,
0) = 0x7f1e7582e000

write(1, "Memory allocated successfully\n", 30Memory allocated successfully
) = 30

write(1, "Block 1 address: 0x7f1e7582f004\n", 32Block 1 address: 0x7f1e7582f004
) = 32

write(1, "Block 2 address: 0x7f1e7582e004\n", 32Block 2 address: 0x7f1e7582e004
) = 32

munmap(0x7f1e7582f000, 68) = 0

munmap(0x7f1e7582e000, 44) = 0

write(1, "Memory freed\n", 13Memory freed
) = 13

munmap(0x7f1e75869000, 512) = 0

write(1, "Program exited successfully\n", 28Program exited successfully
) = 28

exit_group(0) = ?

```

+++ exited with 0 +++

Вывод

В рамках лабораторной работы были реализованы и протестированы два алгоритма управления памятью: алгоритм двойников и алгоритм Мак-Кьюзика-Кэрелса. Оба подхода были интегрированы через общий интерфейс, что позволило сравнить их работу в единых условиях. Buddy Allocator продемонстрировал эффективность при обработке запросов на блоки разного размера, минимизируя фрагментацию. Fixed-Size Allocator показал высокую скорость работы для однотипных запросов благодаря использованию списков свободных блоков. Итогом работы стало понимание принципов управления памятью и выбора подходящего алгоритма в зависимости от задачи.